

Sanger Under Liljen

Supabase (Databasestruktur)

Opprette Supabase-prosjekt:

1. Gå til <https://supabase.com/>, og logg inn eller opprett en konto
2. Gå til “Dashboard” og inn på ønsket organisasjon, eller opprett en ny organisasjon ved å klikke på “New Organization”
3. Lag et nytt prosjekt ved å klikke på “New project”
4. Fyll ut alle feltene og klikk på “Create new project”

Hent nøkler og URL

Prosjektet bruker **Project URL**, **Publishable Key**, og **Service Role Key**.

- **Project URL**

1. I venstremenyen, gå til “Integrations”, og gå til “Data API”
2. Her finner du API URL, Project URL, som vil typisk se slik ut:

`https://abcxyz.supabase.co`

- **Publishable Key** og **Service Role Key**

1. I venstremenyen, gå til “Project Settings” og så “API Keys”
2. Her finner du **Publishable Key**
3. For **Service Role Key**, gå til fanen kalt “Legacy anon, service_role API keys”
4. Service Role Key er skjult (secret), så trykk på “Reveal” for å få tak i nøkkelen

Sett opp database

1. I venstremenyen, gå til “Database” og trykk på “New table”, og fyll inn kolonner for hver tabell

Databasestrukturen for prosjektet:

Litt generell informasjon:

- Alle id-felter er unike primærnøkler, og er tilfeldig generert med “`gen_random_uuid()`”
- On delete cascade = Når en rad i foreldretabellen slettes, blir alle tilknyttede rader i barnetabellen automatisk slettet.
- “created_at”, “submitted_at”, “updated_at”, og “deleted_at” blir alle autogenerert med “`now()`”
- Hvis et felt er obligatorisk, står det (obligatorisk) bak i beskrivelsen av feltet, og på bildet er diamanten til venstre fylt
- Typen på feltet står på høyre side av hvert felt på bildene

- Akkorder i både sanger og sangforslag blir lagret inline i vers og refreng - eks. teksten blir lagret slik: *[Fm]Hei*, der 'Fm' vil bli vist over ordet 'Hei' på nettsiden

Tabell: **songs** - alle sang-objekter

📦 songs	:
♂️ ◆ 🔒 id	uuid
◆ 🔒 title	text
◇ author	text
◇ melody	text
◆ has_chords	bool
◇ created_at	timestampz
◇ updated_at	timestampz
◇ deleted_at	timestampz
◆ 🔒 slug	text
◇ spotify_youtube	text
◇ chorus	text
◆ verses	_text

id: autogenerated (obligatorisk)

title: sangtittel (obligatorisk)

author: forfatter av sang

melody: melodien til sangen

has_chords: basert på om det er lagt til akkorder til sangen eller ikke (obligatorisk)

created_at: tid når sangen ble opprettet

deleted_at: tid når sangen ble slettet

slug: URL-vennlig versjon av tittelen, f.eks. Bålsang 1 => balsang-1 (obligatorisk)

spotify_youtube: link til spotify eller youtube

chorus: refreng

verses: vers - en liste med strenger, *_text* betyr *text* (obligatorisk)

Tabell: **users** - brukere (helst kun speiderledere)

- Alle kan logge inn, men bruker er kun nyttig dersom man er admin eller superuser

📦 users	:
♂️ ◆ 🔒 user_id	uuid
◇ created_at	timestampz
◆ role	text
◇ email	text
◇ name	text

user_id: autogenerated (obligatorisk)

created_at: tid når brukeren ble opprettet

role: rolle - kan være: 'regular', 'admin', eller 'superuser' (obligatorisk)

email: brukerens mail

name: brukerens navn

Tabell: **song_suggestions** - sangforslag sendt inn av speidere som må bli godkjent av speiderledere

song_suggestions		
 	id	uuid
	title	text
	author	text
	melody	text
	status	text
	submitted_at	timestampz
	reviewed_by	uuid
	reviewed_at	timestampz
	spotify_youtube	text
	chorus	text
	verses	_text
	has_chords	bool

id: autogenerated (obligatorisk)

title: sangtittel (obligatorisk)

author: forfatter av sangforslag

melody: melodien til sangforslaget

status: status på om forslaget venter på å bli godkjent/avvist, er godkjent eller avvist. Står alltid som 'pending' og flyttes automatisk til songs-tabellen om godkjent, eller slettes helt om avvist.

submitted_at: tid når sangforslaget ble opprettet

reviewed_by: relasjon til user-tabellen om hvilken bruker/speiderleder som godkjente/avviste forslaget

reviewed_at: tid når sangforslaget ble godkjent/avvist

spotify_youtube: link til spotify eller youtube

chorus: refreng

verses: vers - en liste med strenger, *_text* betyr *text[]* (obligatorisk)

has_chords: basert på om det er lagt til akkorder til sangen eller ikke (obligatorisk)

Tabell: **tags** - kategorier for sanger

tags		
 	id	uuid
 	name	text

id: autogenerated (obligatorisk)

name: navn på kategori, eks. "bålsang" (obligatorisk)

Tabell: **songs-tags** - relasjonstabell mellom “tags” og “songs” for å knytte sanger opp mot kategorier

	song_tags	
	 song_id	uuid
	 tag_id	uuid

song_id: relasjon til ‘id’ i “songs”-tabellen (obligatorisk)

tag_id: relasjon til ‘id’ i “tags”-tabellen (obligatorisk)

Tabell: **playlists** - spillelister med sanger

	playlists	
	 id	uuid
	title	text
	playlist_password	text
	created_at	timestampz
	updated_at	timestampz
	version	int4
	is_public	bool
	expires_at	timestampz

id: autogenerated (obligatorisk)

title: tittel på spilleliste (obligatorisk)

playlist_password: passord på spilleliste for å senere redigere spillelisten dersom den er offentlig (obligatorisk)

created_at: tid når spillelisten ble opprettet (obligatorisk)

updated_at: tid når spillelisten ble oppdatert (obligatorisk)

version: versjon av spilleliste som økes hver gang den oppdateres med nytt innhold

is_public: om spilleliste er offentlig eller ikke (kan sløyfes da private spillelister kun er lagret i

IndexedDB)

expires_at: tid når spillelisten utløper (kun offentlige) for så å slettes, for å hindre at databasen er fylt av gamle spillelister (obligatorisk)

Tabell: **playlist_items** - relasjonstabell mellom “songs” og “playlists”, hva slags sanger en spilleliste inneholder.

	playlist_items	
	 playlist_id	uuid
	 song_id	uuid
	position	int4

playlist_id: foreign key til ‘id’ i “playlists”-tabellen (obligatorisk) - for playlist_id, så må “Action if referenced row is removed” settes som “cascade”

song_id: foreign key til 'id' i "songs"-tabellen (obligatorisk)

position: posisjonen på sangen i spillelisten for å holde rekkefølgen på sangene

RPC: "Functions" og "triggers"

I prosjektet har vi satt opp både vanlig REST-API og RPC fordi de løser ulike behov. Vanlige API-kall mot Supabase (for eksempel `GET /playlists` eller `POST /songs`) er ok til enkle operasjoner som å hente eller opprette data direkte i en tabell. De fungerer godt når vi ikke trenger spesiell logikk, men bare vil lese eller skrive data.

RPC bruker vi der vi trenger mer kontroll over hva som faktisk skjer i databasen. I stedet for å gi direkte tilgang til å oppdatere eller slette rader, lager vi en databasefunksjon som først sjekker nødvendige betingelser (for eksempel at riktig passord er oppgitt), og deretter utfører handlingen. På den måten ligger sikkerhetslogikken i databasen, ikke bare i frontend eller i API-laget. Selv om noen prøver å bruke Postman eller kalle Supabase direkte, vil operasjonen bli stoppet dersom kravene i funksjonen ikke oppfylles.

Fordelen med denne løsningen er at vi får:

- Bedre sikkerhet: Vi kan blokkere direkte endringer via RLS og kun tillate endringer gjennom kontrollerte funksjoner.
- Samlet forretningslogikk: Flere steg (validering, oppdatering av versjon, sletting osv.) kan skje i én transaksjon.
- Mindre risiko for misbruk: Reglene håndheves i databasen, uavhengig av klient.

Du finner dem i venstremenyen under "Database" -> "Functions"/"Triggers"

Alle funksjoner (Definisjon ("Definiton") ligger i raden under):

Navn	Argumenter	Return type	Hva funksjonen gjør
cleanup_expired_playlists	None	void	Sletter spillelister som har utløpt.
<pre>begin delete from playlists where expires_at is not null and expires_at < now();</pre>			

<pre> return; end; </pre>			
handle_new_admin_user	None	trigger	Oppretter bruker i admin_users. Setter rolle til regular. Gjør ingenting hvis brukeren finnes fra før. Returnerer brukeren. (Funksjonen refererer til admin fordi alle innloggede brukere skal i prinsippet kun være admin eller superuser)
<pre> begin insert into public.admin_users (user_id, role) values (new.id, 'regular') on conflict (user_id) do nothing; return new; end; </pre>			
handle_new_user	None	trigger	Oppretter bruker i users-tabellen. Oppdaterer bruker hvis den finnes fra før. Lagrer e-post og navn. Returnerer brukeren.
<pre> begin insert into public.users (user_id, email, name) values (new.id, new.email, coalesce(new.raw_user_meta_data->>'full_name', new.raw_user_meta_data->>'name', new.email)) on conflict (user_id) do update set </pre>			

```

    email = excluded.email,
    name = excluded.name;

    return new;
end;
```

playlists_create

p_title
p_password
p_is_public
p_expires_at

playlists

Oppretter en ny spilleliste

Hash'er passordet med
`crypt() + gen_salt('bf')`

Returnerer den opprettede raden

```

declare
    v_row public.playlists;
begin
    insert into public.playlists (title, playlist_password, is_public, expires_at)
    values (p_title, crypt(p_password, gen_salt('bf')), p_is_public, p_expires_at)
    returning * into v_row;

    return v_row;
end;
```

playlists_add_item

p_playlist_id
p_password
p_song_id

void

Sjekker at passordet stemmer

Legger til en sang i
`playlist_items`

Setter riktig `position`

Øker `version` på spillelisten

```

declare
    v_hash text;
    v_next_pos int;
begin
    select playlist_password into v_hash
    from public.playlists
    where id = p_playlist_id;

    if v_hash is null or crypt(p_password, v_hash) <> v_hash then
        raise exception 'Invalid password';
    end if;
end;
```

```

end if;

select coalesce(max(position), 0) + 1
into v_next_pos
from public.playlist_items
where playlist_id = p_playlist_id;

insert into public.playlist_items (playlist_id, song_id, position)
values (p_playlist_id, p_song_id, v_next_pos)
on conflict (playlist_id, song_id) do update
    set position = excluded.position;

update public.playlists set version = version + 1
where id = p_playlist_id;
end;

```

playlists_admin_add_
item

p_playlist_id
p_song_id

void

Legger til en sang i en spilleliste
som admin.

```

declare
    v_next_pos int;
begin
    select coalesce(max(position), 0) + 1
    into v_next_pos
    from public.playlist_items
    where playlist_id = p_playlist_id;

    insert into public.playlist_items (playlist_id, song_id, position)
    values (p_playlist_id, p_song_id, v_next_pos)
    on conflict (playlist_id, song_id) do update
        set position = excluded.position;

    update public.playlists
    set version = version + 1
    where id = p_playlist_id;
end;

```

playlists_remove_ite
m

p_playlist_id
p_password
p_song_id

void

Sjekker passord

Fjerner sang fra
playlist_items

			Øker version
<pre> declare v_row public.playlists; begin update public.playlists set title = p_title, is_public = p_is_public, expires_at = p_expires_at, playlist_password = case when p_new_password is not null and btrim(p_new_password) <> '' then crypt(p_new_password, gen_salt('bf')) else playlist_password end, version = version + 1, updated_at = now() where id = p_playlist_id returning * into v_row; return v_row; end;</pre>			
playlists_admin_remove_item	p_playlist_id p_song_id	void	Fjerner sanger fra en spilleliste som admin.
<pre> begin delete from public.playlist_items where playlist_id = p_playlist_id and song_id = p_song_id; update public.playlists set version = version + 1 where id = p_playlist_id; end;</pre>			
playlists_delete	p_playlist_id p_password	void	Sjekker passord Sletter spillelisten (og cascading sletter items)
<pre> declare</pre>			

```

v_hash text;
begin
    select playlist_password into v_hash
    from public.playlists
    where id = p_playlist_id;

    if v_hash is null or crypt(p_password, v_hash) <> v_hash then
        raise exception 'Invalid password';
    end if;

    delete from public.playlists
    where id = p_playlist_id;
end;

```

playlists_admin_delete	p_playlist_id	void	Sletter spilleliste som admin.
------------------------	---------------	------	--------------------------------

```

begin
    delete from public.playlists
    where id = p_playlist_id;
end;

```

playlists_update	p_playlist_id p_current_password p_new_password p_title p_is_public p_expires_at	playlists	Sjekker passord. Oppdaterer spillelisten. Endrer passord hvis oppgitt. Returnerer oppdatert spilleliste.
------------------	---	-----------	---

```

declare
    v_hash text;
    v_row public.playlists;
begin
    select playlist_password into v_hash
    from public.playlists
    where id = p_playlist_id;

    if v_hash is null or crypt(p_current_password, v_hash) <> v_hash then
        raise exception 'Invalid password';
    end if;

    update public.playlists
    set
        title = p_title,

```

```

is_public = p_is_public,
expires_at = p_expires_at,
playlist_password = case
    when p_new_password is not null and btrim(p_new_password) <> ''
    then crypt(p_new_password, gen_salt('bf'))
    else playlist_password
end,
version = version + 1,
updated_at = now()
where id = p_playlist_id
returning * into v_row;

return v_row;
end;

```

playlists_admin_update

p_playlist_id
p_new_password
p_title
p_is_public
p_expires_at

playlists

Sjekker passordet for spillelisten.

Hvis passordet er feil: stopper med feil.

Hvis riktig: fjerner sangen fra spillelisten.

Oppdaterer versjonsnummeret på spillelisten.

```

declare
    v_hash text;
begin
    select playlist_password into v_hash
    from public.playlists
    where id = p_playlist_id;

    if v_hash is null or crypt(p_password, v_hash) <> v_hash then
        raise exception 'Invalid password';
    end if;

    delete from public.playlist_items
    where playlist_id = p_playlist_id and song_id = p_song_id;

    update public.playlists set version = version + 1
    where id = p_playlist_id;
end;

```

playlists_get_detail	p_playlist_id	alle attributter til spilleliste	Henter alle detaljer for en spilleliste
<pre> select p.id, p.title, p.is_public, p.expires_at, p.created_at, p.updated_at, p.version, (p.playlist_password is not null and btrim(p.playlist_password) <> '') as has_password from public.playlists p where p.id = p_playlist_id; </pre>			
playlists_list_public	None	alle attributter til spilleliste	Henter alle detaljene til spillelistene som er offentlig
<pre> select p.id, p.title, p.created_at, p.updated_at, p.version, p.is_public, p.expires_at, (p.playlist_password is not null and btrim(p.playlist_password) <> '') as has_password from public.playlists p where p.is_public = true; </pre>			
playlists_verify_password	p_playlist_id p_password	boolean	Sjekker at passordet til en spilleliste stemmer med brukerens input
<pre> declare v_hash text; begin select playlist_password into v_hash from public.playlists where id = p_playlist_id; if v_hash is null then return false; </pre>			

<pre> end if; return crypt(p_password, v_hash) = v_hash; end;</pre>			
set_updated_at	None	trigger	Setter tid for oppdatering av objekt
<pre> begin new.updated_at = now(); return new; end;</pre>			
slufigy	input	text	
<pre> declare result text; begin -- Custom replacements first result := lower(input); result := replace(result, 'æ', 'ea'); result := replace(result, 'ø', 'o'); result := replace(result, 'å', 'aa'); -- Replace anything not a-z or 0-9 with dash result := regexp_replace(result, '^[a-z0-9]+', '-', 'g'); -- Trim leading/trailing dashes result := trim(both '-' from result); return result; end;</pre>			

Alle triggers:

trg_playlists_updated_at

- Returnert fra funksjonen 'set_updated_at' for playlists

trg_songs_updated_at

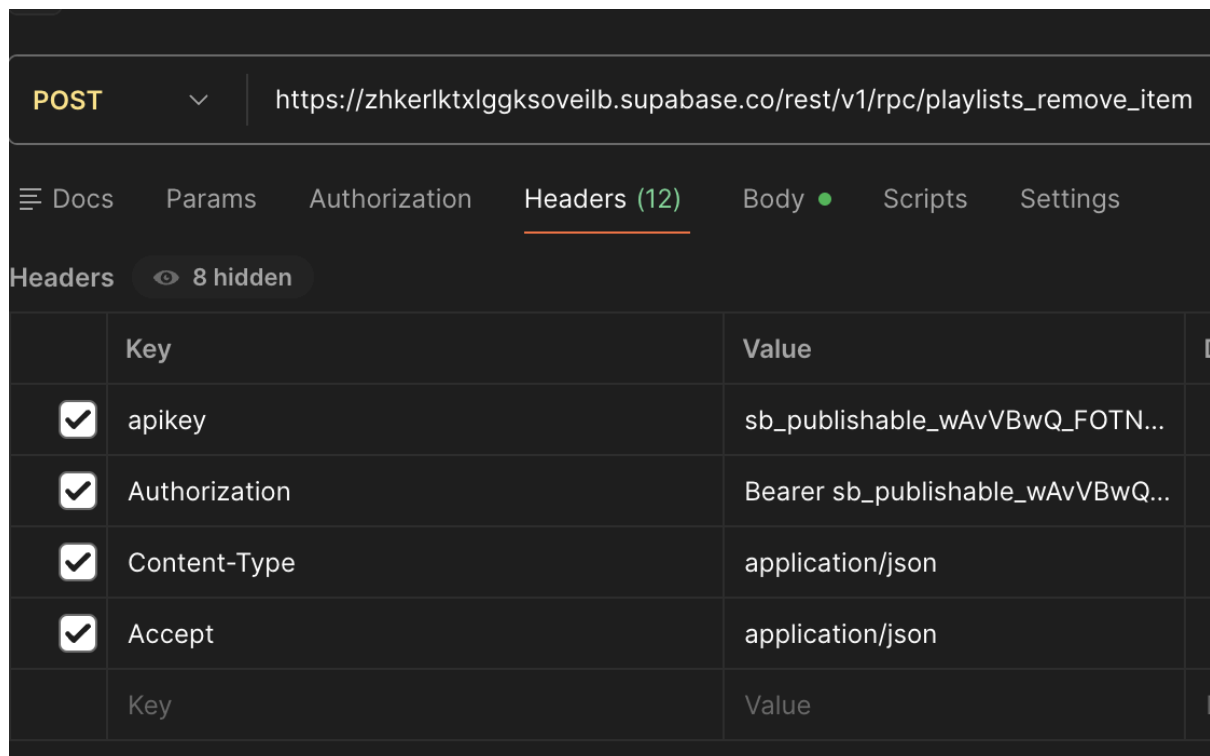
- Returnert fra funksjonen 'set_updated_at' for songs

NB! Oppsett av lokal database finnes i /frontend/src/lib/[db.ts](#). Her finnes favoritter som kun lagres lokalt.

Hvis du ønsker å teste RPC:

For å teste RPC-løsningen selv kan dere bruke Postman og sammenligne direkte REST-kall med RPC-kall:

Før du trykker “SEND” på en request, fyll inn i header:



sb_publishable er hele den publishable nøkkelen som ligger i Supabase-prosjektet.

Test at direkte sletting er blokkert (sikkerhet)

Prøv å sende en **DELETE** mot

`https://zhkerlktxlggksoveilb.supabase.co/rest/v1/playlists?id=eq.<playlist_id>`

med anon-nøkkelen i header.

Dere skal få **401/403**. Det betyr at RLS blokkerer direkte endringer.

Test at RPC fungerer med riktig passord

Send en **POST** til

https://zhkerlktxlggksoveilb.supabase.co/rest/v1/playlists?id=eq.<playlist_id>

med body:

```
{
  "p_playlist_id": "<playlist_id>",
  "p_password": "1234"
}
```

Autentisering

Sette opp Google Cloud:

Vi bruker Google Auth for autentisering

1. Gå til Google Cloud Console og logg inn
2. I venstremenyen klikk på "APIs & Services", og velg "New Project"
3. Fyll inn nødvendige felter og opprett prosjekt
4. Gå til "APIs & Services" -> "OAuth consent screen"
5. Du skal nå være på "Overview"
6. Trykk på "Get started", fyll ut felter, og trykk på "Create"
7. På "Overview" trykk på "Create OAuth Client", og fyll ut alle felter
8. Under "Clients" finner du "Client ID" og "Client secret"
 - "Client secret" finnes under redigering på høyre side (penn-ikon)

OAuth 2.0 Client IDs

☐	Name	Creation date ↓	Type	Client ID	Actions
☐	Bachelor_G01	Feb 28, 2026	Web application	710878200978-sbe6...	

Sette opp Supabase Auth med Google:

1. I venstremenyen gå til "Authentication", og så klikk på "Sign in / Providers"
2. Bla ned til "Google", enable, og fyll inn "Client ID" og "Client secret", og lagre
3. I venstremenyen klikk på "Authentication", og så "URL Configuration" og lim inn domenet du kjører prosjektet på i "Site URL"

Frontend

1. Gå til GitHub og klon repository: <https://github.com/1-fredrikstad/1sang/>
2. Åpne opp prosjektet på PC (eks. Visual Studio Code)
3. I “frontend”-mappen, lag en ny fil og kall den: “.env.local”
4. Lim inn:

```
NEXT_PUBLIC_SUPABASE_URL='<your_supabase_project_url>'
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY='<your_supabase_public_key>'
SUPABASE_SERVICE_ROLE_KEY='<your_service_role_key>'
```

- **NB! Erstatt alt i hermetegn med faktiske verdier fra Supabase gitt over**

5. Installer packages med “pnpm”

```
pnpm install
```

6. Åpne opp terminalen og gå inn i “frontend”-mappen

```
cd frontend
```

Bygge prosjekt for production, med service worker:

7. Bygg prosjektet

```
pnpm build
```

8. Start prosjektet

```
pnpm start
```

Kjøre prosjekt i development-mode:

9. Start development server

```
pnpm dev
```

OBS! For å hindre Supabase fra å pause (etter 1 uke av inaktivitet), en GitHub Action må settes opp:

- I mappen: `./github/workflows/keep-alive.yml`, er det en jobb som er ment å kjøre hver tredje dag. Denne jobben skal treffe endepunktet

<https://sangerunderliljen.vercel.app/api/health>, for å holde Supabase kjørende (simulert aktivitet).

- I “/app/api/health” er det en HTTP GET request som henter data fra Supabase (tilfeldigvis valgt “id” fra “songs” tabellen), og loggfører aktiviteten for å sikre at jobben kjører hver tredje dag.
- Dette sjekkes i Vercel-loggen (hvis det er denne hosting-plattformen som brukes), og burde bli sjekket jevnlig for å være sikker på at Supabase ikke pauser.

Vercel - hvis det ønskes å hoste selv

1. Gå til vercel: <https://vercel.com/> og logg inn (eller opprett bruker om du ikke har en enda)
2. Koble til GitHub-kontoen din hvis du ikke allerede har gjort det
3. Gå inn på “Projects” venstremenyen, klikk på “Add New...” og så “Project”
4. Velg repo, i dette tilfellet “1-fredrikstad”, og velg repoet du ønsker å hoste, her “1-sang”, og trykk på “import”
5. Sjekk build/settings
6. Legg til **Environment Variables** som er gitt over fra Supabase
7. Klikk **Deploy**, og åpne linken Vercel gir deg etter build har fullført
8. Hvis du ønsker eget domene, gå til Project Settings -> Domains